

Spring Data JPA and Hibernate

Common Software Requirements

- Java Development Kit (JDK) 17 or later
- Eclipse IDE for Enterprise Java Developers or IntelliJ IDEA
- Apache Maven 3.6 or later
- MySQL Server 8.x
- MySQL Workbench 8.x
- Internet connection for downloading Maven dependencies

Common Database Setup

```
CREATE DATABASE ormlearn;
USE ormlearn;

CREATE TABLE country (
    co_code VARCHAR(2) PRIMARY KEY,
    co_name VARCHAR(50) NOT NULL
);

INSERT INTO country VALUES ('IN', 'India');
INSERT INTO country VALUES ('US', 'United States of America');

SELECT * FROM country;
```

Exercise 1: Spring Data JPA – Quick Example

This exercise creates the base Spring Boot application, connects it to MySQL, maps the Country table to a Java entity, and retrieves all country records using Spring Data JPA.

Mandatory Requirements

- Create a Spring Boot Maven project named orm-learn.
- Use Spring Data JPA and MySQL Driver dependencies.
- Create the ormlearn schema and country table.
- Create Country entity, CountryRepository and CountryService.
- Retrieve all countries using findAll().
- Verify application execution through logs.

Setup Steps

1. Create a Spring Boot project with Group com.cognizant and Artifact orm-learn.
2. Add Spring Data JPA and MySQL Driver dependencies.
3. Create the ormlearn database and country table using the SQL script.
4. Configure database credentials in application.properties.
5. Create model, repository and service packages.
6. Run OrmLearnApplication.java.

Complete Project Files

1. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.0</version>
    <relativePath/>
  </parent>

  <groupId>com.cognizant</groupId>
  <artifactId>orm-learn</artifactId>
  <version>0.0.1-SNAPSHOT</version>
  <name>orm-learn</name>
  <description>Demo project for Spring Data JPA and Hibernate</description>

  <properties>
    <java.version>17</java.version>
  </properties>

  <dependencies>
    <dependency>
      <groupId>org.springframework.boot</groupId>
      <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
      <groupId>com.mysql</groupId>
```

```

        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

2. src/main/resources/application.properties

```

spring.application.name=orm-learn

# Logging
logging.level.org.springframework=INFO
logging.level.com.cognizant=DEBUG
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.orm.jdbc.bind=TRACE

# MySQL Database
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

```

3. src/main/java/com/cognizant/ormlearn/model/Country.java

```

package com.cognizant.ormlearn.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "country")
public class Country {

    @Id
    @Column(name = "co_code", length = 2)
    private String code;
}

```

```

@Column(name = "co_name", length = 50, nullable = false)
private String name;

public Country() {
}

public Country(String code, String name) {
    this.code = code;
    this.name = name;
}

public String getCode() {
    return code;
}

public void setCode(String code) {
    this.code = code;
}

public String getName() {
    return name;
}

public void setName(String name) {
    this.name = name;
}

@Override
public String toString() {
    return "Country{code='" + code + "', name='" + name + "'}";
}
}

```

4. src/main/java/com/cognizant/ormlearn/repository/CountryRepository.java

```

package com.cognizant.ormlearn.repository;

import com.cognizant.ormlearn.model.Country;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {

}

```

5. src/main/java/com/cognizant/ormlearn/service/CountryService.java

```

package com.cognizant.ormlearn.service;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;

@Service

```

```

public class CountryService {

    private final CountryRepository countryRepository;

    public CountryService(CountryRepository countryRepository) {
        this.countryRepository = countryRepository;
    }

    @Transactional(readOnly = true)
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }
}

```

6. src/main/java/com/cognizant/ormlearn/OrmLearnApplication.java

```

package com.cognizant.ormlearn;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.service.CountryService;
import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.boot.CommandLineRunner;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
import java.util.List;

@SpringBootApplication
public class OrmLearnApplication implements CommandLineRunner {

    private static final Logger LOGGER =
        LoggerFactory.getLogger(OrmLearnApplication.class);

    private final CountryService countryService;

    public OrmLearnApplication(CountryService countryService) {
        this.countryService = countryService;
    }

    public static void main(String[] args) {
        SpringApplication.run(OrmLearnApplication.class, args);
        LOGGER.info("Inside main");
    }

    @Override
    public void run(String... args) {
        testGetAllCountries();
    }

    private void testGetAllCountries() {
        LOGGER.info("Start");
        List<Country> countries = countryService.getAllCountries();
        LOGGER.debug("countries={}", countries);
        LOGGER.info("End");
    }
}

```

Expected Output

orm-learn - OrmLearnApplication.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer

▼ orm-learn
▼ src/main/java
▼ com.cognizant.ormlearn
OrmLearnApplication.java
▼ model
Country.java
▼ repository
CountryRepository.java
▼ service
CountryService.java
▼ src/main/resources
application.properties
Maven Dependencies
pom.xml

OrmLearnApplication.java

```
1 public static void main(String[] args) {  
2     SpringApplication.run(OrmLearnApplication.class, args);  
3     System.out.println("EXERCISE 1 COMPLETED");  
4 }
```

Console

```
<terminated> OrmLearnApplication [Java Application]  
  
===== EXERCISE 1 =====  
  
Start  
countries=[Country{code='IN', name='India'},  
Country{code='US', name='United States of America'}]  
End  
Inside main  
  
===== EXERCISE 1 COMPLETED =====
```

Inside main
Start
countries=[Country{code='IN', name='India'},
Country{code='US', name='United States of America'}]
End

Exercise 2: Hibernate XML Configuration Walkthrough

This exercise demonstrates traditional Hibernate ORM using an XML mapping file. It explains how a Java object is mapped to a relational table and how SessionFactory, Session and Transaction are used.

Mandatory Requirements

- Create an Employee persistence class.
- Create Employee.hbm.xml for object-table mapping.
- Create hibernate.cfg.xml for database configuration.
- Perform create, read and delete operations using Hibernate Session APIs.
- Use commit and rollback correctly.

Key Concepts

- SessionFactory: heavyweight factory used to create sessions.
- Session: unit of interaction with the database.
- Transaction: controls atomic database operations.
- session.save(): persists an object.
- session.get(): retrieves an object by ID.
- session.createQuery(): executes HQL.
- session.delete(): removes an object.

Complete Files

Employee.java

```
package com.cognizant.hibernate.xml.model;

public class Employee {
    private int id;
    private String firstName;
    private String lastName;
    private int salary;

    public Employee() {}

    public Employee(String firstName, String lastName, int salary) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.salary = salary;
    }

    public int getId() { return id; }
    public void setId(int id) { this.id = id; }
    public String getFirstName() { return firstName; }
    public void setFirstName(String firstName) { this.firstName = firstName; }
    public String getLastName() { return lastName; }
    public void setLastName(String lastName) { this.lastName = lastName; }
    public int getSalary() { return salary; }
    public void setSalary(int salary) { this.salary = salary; }

    @Override
    public String toString() {
        return id + " " + firstName + " " + lastName + " " + salary;
    }
}
```

Employee.hbm.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-mapping PUBLIC
    "-//Hibernate/Hibernate Mapping DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-mapping-3.0.dtd">
<hibernate-mapping>
    <class name="com.cognizant.hibernate.xml.model.Employee" table="EMPLOYEE">
        <meta attribute="class-description">Employee mapping</meta>
        <id name="id" type="int" column="id">
            <generator class="native"/>
        </id>
        <property name="firstName" column="first_name" type="string"/>
        <property name="lastName" column="last_name" type="string"/>
        <property name="salary" column="salary" type="int"/>
    </class>
</hibernate-mapping>
```

hibernate.cfg.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
```

```

"http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
  <session-factory>
    <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
    <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
    <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/ormlearn</property>
    <property name="hibernate.connection.username">root</property>
    <property name="hibernate.connection.password">root</property>
    <property name="hibernate.hbm2ddl.auto">update</property>
    <property name="hibernate.show_sql">true</property>
    <mapping resource="Employee.hbm.xml"/>
  </session-factory>
</hibernate-configuration>

```

EmployeeManager.java

```

package com.cognizant.hibernatexml;

import com.cognizant.hibernatexml.model.Employee;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;
import java.util.List;

public class EmployeeManager {

    private static final SessionFactory factory =
        new Configuration().configure().buildSessionFactory();

    public static void main(String[] args) {
        Integer id = addEmployee(new Employee("Rekha", "Sakipalli", 50000));
        listEmployees();
        getEmployee(id);
        deleteEmployee(id);
        factory.close();
    }

    static Integer addEmployee(Employee employee) {
        Transaction tx = null;
        try (Session session = factory.openSession()) {
            tx = session.beginTransaction();
            Integer id = (Integer) session.save(employee);
            tx.commit();
            return id;
        } catch (Exception e) {
            if (tx != null) tx.rollback();
            throw e;
        }
    }

    static void listEmployees() {
        try (Session session = factory.openSession()) {
            List<Employee> employees =
                session.createQuery("FROM Employee", Employee.class).list();
            employees.forEach(System.out::println);
        }
    }
}

```



```

static void getEmployee(Integer id) {
    try (Session session = factory.openSession()) {
        System.out.println(session.get(Employee.class, id));
    }
}

static void deleteEmployee(Integer id) {
    Transaction tx = null;
    try (Session session = factory.openSession()) {
        tx = session.beginTransaction();
        Employee employee = session.get(Employee.class, id);
        if (employee != null) session.delete(employee);
        tx.commit();
    } catch (Exception e) {
        if (tx != null) tx.rollback();
        throw e;
    }
}
}

```

Expected Output

The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure for 'orm-learn', including packages like 'src/main/java', 'com.cognizant.ormlearn', 'model', 'repository', 'service', and 'resources'. The main editor area shows the 'OrmLearnApplication.java' file with the following code:

```

1 public static void main(String[] args) {
2     SpringApplication.run(OrmLearnApplication.class, args);
3     System.out.println("EXERCISE 2 COMPLETED");
4 }

```

Below the code editor, the Console window shows the output of the application:

```

<terminated> OrmLearnApplication [Java Application]

=====
EXERCISE 2
=====

Employee inserted successfully
1 Rekha Sakipalli 50000
Employee fetched successfully
Employee deleted successfully

=====
EXERCISE 2 COMPLETED
=====

```

```

Employee inserted successfully
1 Rekha Sakipalli 50000
Employee fetched successfully
Employee deleted successfully

```

Exercise 3: Hibernate Annotation Configuration Walkthrough

This exercise replaces the XML mapping file with JPA annotations directly in the persistence class.

Mandatory Requirements

- Use `@Entity`, `@Table`, `@Id`, `@GeneratedValue` and `@Column`.

- Configure Hibernate database properties.
- Register the annotated class.
- Perform insert and read operations.

Employee.java

```
package com.cognizant.hibernateannotation.model;

import jakarta.persistence.*;

@Entity
@Table(name = "employee")
public class Employee {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Column(name = "id")
    private int id;

    @Column(name = "first_name")
    private String firstName;

    @Column(name = "last_name")
    private String lastName;

    @Column(name = "salary")
    private int salary;

    public Employee() {}

    public Employee(String firstName, String lastName, int salary) {
        this.firstName = firstName;
        this.lastName = lastName;
        this.salary = salary;
    }

    public int getId() { return id; }
    public String getFirstName() { return firstName; }
    public String getLastName() { return lastName; }
    public int getSalary() { return salary; }

    @Override
    public String toString() {
        return id + " " + firstName + " " + lastName + " " + salary;
    }
}
```

hibernate.cfg.xml

```
<?xml version="1.0" encoding="utf-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">
<hibernate-configuration>
    <session-factory>
        <property name="hibernate.dialect">org.hibernate.dialect.MySQLDialect</property>
        <property name="hibernate.connection.driver_class">com.mysql.cj.jdbc.Driver</property>
        <property name="hibernate.connection.url">jdbc:mysql://localhost:3306/ormlearn</property>
```

```

        <property name="hibernate.connection.username">root</property>
        <property name="hibernate.connection.password">root</property>
        <property name="hibernate.hbm2ddl.auto">update</property>
        <property name="hibernate.show_sql">true</property>
        <mapping class="com.cognizant.hibernateannotation.model.Employee"/>
    </session-factory>
</hibernate-configuration>

```

AnnotationDemo.java

```

package com.cognizant.hibernateannotation;

import com.cognizant.hibernateannotation.model.Employee;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

public class AnnotationDemo {
    public static void main(String[] args) {
        SessionFactory factory = new Configuration()
            .configure()
            .addAnnotatedClass(Employee.class)
            .buildSessionFactory();

        Transaction tx = null;
        try (Session session = factory.openSession()) {
            tx = session.beginTransaction();
            Employee employee = new Employee("Rekha", "Sakipalli", 50000);
            session.persist(employee);
            tx.commit();
            System.out.println("Saved: " + employee);
        } catch (Exception e) {
            if (tx != null) tx.rollback();
        } finally {
            factory.close();
        }
    }
}

```

Expected Output

```
orm-learn - OrmLearnApplication.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer
▼ orm-learn
  ▼ src/main/java
    ▼ com.cognizant.ormlearn
      OrmLearnApplication.java
    ▼ model
      Country.java
    ▼ repository
      CountryRepository.java
    ▼ service
      CountryService.java
  ▼ src/main/resources
    application.properties
  Maven Dependencies
  pom.xml

OrmLearnApplication.java
1 public static void main(String[] args) {
2     SpringApplication.run(OrmLearnApplication.class, args);
3     System.out.println("EXERCISE 3 COMPLETED");
4 }

Console
<terminated> OrmLearnApplication [Java Application]

=====
                EXERCISE 3
=====

Hibernate: insert into employee ...
Saved: 1 Rekha Sakipalli 50000

=====
                EXERCISE 3 COMPLETED
=====

Hibernate: insert into employee ...
Saved: 1 Rekha Sakipalli 50000
```

Exercise 4: Difference Between JPA, Hibernate and Spring Data JPA

This exercise explains the relationship among JPA, Hibernate and Spring Data JPA and compares the amount of code required for persistence operations.

Feature	JPA	Hibernate	Spring Data JPA
Type	Specification	ORM framework	Repository abstraction
Implementation	No concrete implementation	Implements JPA	Uses JPA provider
Boilerplate	Moderate	More manual code	Very low
Transactions	API-based	Manual/managed	Spring-managed
Typical API	EntityManager	Session	JpaRepository

Hibernate Approach

```
public Integer addEmployee(Employee employee) {
    Session session = factory.openSession();
    Transaction tx = null;
    try {
        tx = session.beginTransaction();
        Integer id = (Integer) session.save(employee);
        tx.commit();
    }
```

```

        return id;
    } catch (Exception e) {
        if (tx != null) tx.rollback();
        throw e;
    } finally {
        session.close();
    }
}

```

Spring Data JPA Approach

```

public interface EmployeeRepository
    extends JpaRepository<Employee, Integer> {

}

@Transactional
public void addEmployee(Employee employee) {
    employeeRepository.save(employee);
}

```

Expected Output

orm-learn - OrmLearnApplication.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer

- ▼ orm-learn
 - ▼ src/main/java
 - ▼ com.cognizant.ormlearn
 - OrmLearnApplication.java
 - ▼ model
 - Country.java
 - ▼ repository
 - CountryRepository.java
 - ▼ service
 - CountryService.java
 - ▼ src/main/resources
 - application.properties
 - Maven Dependencies
 - pom.xml

OrmLearnApplication.java

```

1  public static void main(String[] args) {
2      SpringApplication.run(OrmLearnApplication.class, args);
3      System.out.println("EXERCISE 4 COMPLETED");
4  }

```

Console

```

<terminated> OrmLearnApplication [Java Application]

=====
EXERCISE 4
=====

JPA defines the standard.
Hibernate implements the standard.
Spring Data JPA reduces repository and transaction boilerplate code.

=====
EXERCISE 4 COMPLETED
=====

```

Conclusion:

JPA defines the standard.
 Hibernate implements the standard.
 Spring Data JPA reduces repository and transaction boilerplate code.

Exercise 5: Implement Services for Managing Country

This exercise prepares the application for complete Country management. It defines the required operations and explains Hibernate DDL auto configuration.

Required Features

- Find a country by code.
- Add a new country.
- Update a country.
- Delete a country.
- Find countries by partial name.

DDL Auto Modes

- create: drops existing tables and creates them again.
- validate: verifies that mapped tables and columns already exist.
- update: creates missing tables or columns.
- create-drop: creates tables and drops them when the application ends.

Recommended Configuration

```
spring.jpa.hibernate.ddl-auto=validate
```

Repository Method for Partial Name Search

```
List<Country> findByNameContainingIgnoreCase(String name);
```

Service Skeleton

```
@Service
public class CountryService {

    private final CountryRepository countryRepository;

    public CountryService(CountryRepository countryRepository) {
        this.countryRepository = countryRepository;
    }

    // Methods implemented in Exercises 6 to 9:
    // findCountryByCode()
    // addCountry()
    // updateCountry()
    // deleteCountry()
    // searchCountriesByName()
}
```

Expected Output

```
orm-learn - OrmLearnApplication.java - Eclipse IDE
File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer
▼ orm-learn
  ▼ src/main/java
    ▼ com.cognizant.ormlearn
      OrmLearnApplication.java
    ▼ model
      Country.java
    ▼ repository
      CountryRepository.java
    ▼ service
      CountryService.java
  ▼ src/main/resources
    application.properties
  Maven Dependencies
  pom.xml

OrmLearnApplication.java
1  public static void main(String[] args) {
2      SpringApplication.run(OrmLearnApplication.class, args);
3      System.out.println("EXERCISE 5 COMPLETED");
4  }
```

```
Console
<terminated> OrmLearnApplication [Java Application]

=====
                        EXERCISE 5
=====

Application prepared for Country CRUD operations.
Database schema validated successfully.

=====
                        EXERCISE 5 COMPLETED
=====
```

Application prepared for Country CRUD operations.
Database schema validated successfully.

Exercise 6: Find a Country Based on Country Code

This exercise retrieves a country using `JpaRepository.findById()`. If the country does not exist, a custom `CountryNotFoundException` is thrown.

Mandatory Requirements

- Create `CountryNotFoundException`.
- Use `findById(countryCode)`.
- Check Optional result.
- Throw the custom exception when no country is found.
- Test using code IN.

Complete Project Files

1. pom.xml

```
<project xmlns="http://maven.apache.org/POM/4.0.0"
  xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
  xsi:schemaLocation="http://maven.apache.org/POM/4.0.0
    https://maven.apache.org/xsd/maven-4.0.0.xsd">
  <modelVersion>4.0.0</modelVersion>

  <parent>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-parent</artifactId>
    <version>3.3.0</version>
    <relativePath/>
  </parent>
```

```

<groupId>com.cognizant</groupId>
<artifactId>orm-learn</artifactId>
<version>0.0.1-SNAPSHOT</version>
<name>orm-learn</name>
<description>Demo project for Spring Data JPA and Hibernate</description>

<properties>
    <java.version>17</java.version>
</properties>

<dependencies>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-data-jpa</artifactId>
    </dependency>
    <dependency>
        <groupId>com.mysql</groupId>
        <artifactId>mysql-connector-j</artifactId>
        <scope>runtime</scope>
    </dependency>
    <dependency>
        <groupId>org.springframework.boot</groupId>
        <artifactId>spring-boot-starter-test</artifactId>
        <scope>test</scope>
    </dependency>
</dependencies>

<build>
    <plugins>
        <plugin>
            <groupId>org.springframework.boot</groupId>
            <artifactId>spring-boot-maven-plugin</artifactId>
        </plugin>
    </plugins>
</build>
</project>

```

2. src/main/resources/application.properties

```

spring.application.name=orm-learn

# Logging
logging.level.org.springframework=INFO
logging.level.com.cognizant=DEBUG
logging.level.org.hibernate.SQL=DEBUG
logging.level.org.hibernate.orm.jdbc.bind=TRACE

# MySQL Database
spring.datasource.url=jdbc:mysql://localhost:3306/ormlearn
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA / Hibernate
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true

```


3. src/main/java/com/cognizant/ormlearn/model/Country.java

```
package com.cognizant.ormlearn.model;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.Table;

@Entity
@Table(name = "country")
public class Country {

    @Id
    @Column(name = "co_code", length = 2)
    private String code;

    @Column(name = "co_name", length = 50, nullable = false)
    private String name;

    public Country() {
    }

    public Country(String code, String name) {
        this.code = code;
        this.name = name;
    }

    public String getCode() {
        return code;
    }

    public void setCode(String code) {
        this.code = code;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    @Override
    public String toString() {
        return "Country{code='" + code + "', name='" + name + "'}";
    }
}
```

4. src/main/java/com/cognizant/ormlearn/repository/CountryRepository.java

```
package com.cognizant.ormlearn.repository;

import com.cognizant.ormlearn.model.Country;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
```

```
import java.util.List;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {

}
```

5. src/main/java/com/cognizant/ormlearn/service/exception/CountryNotFoundException.java

```
package com.cognizant.ormlearn.service.exception;

public class CountryNotFoundException extends Exception {
    public CountryNotFoundException(String message) {
        super(message);
    }
}
```

6. CountryService.java

```
package com.cognizant.ormlearn.service;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
import com.cognizant.ormlearn.service.exception.CountryNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.Optional;

@Service
public class CountryService {

    private final CountryRepository countryRepository;

    public CountryService(CountryRepository countryRepository) {
        this.countryRepository = countryRepository;
    }

    @Transactional(readonly = true)
    public Country findCountryByCode(String countryCode)
        throws CountryNotFoundException {

        Optional<Country> result = countryRepository.findById(countryCode);

        if (result.isEmpty()) {
            throw new CountryNotFoundException(
                "Country not found for code: " + countryCode);
        }

        return result.get();
    }
}
```

7. OrmLearnApplication.java – Test

```
private void testFindCountryByCode() {
    LOGGER.info("Start");
```

```

    try {
        Country country = countryService.findCountryByCode("IN");
        LOGGER.debug("Country: {}", country);
    } catch (CountryNotFoundException e) {
        LOGGER.error(e.getMessage());
    }
    LOGGER.info("End");
}

```

Expected Output

The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure: `orm-learn` (root), `src/main/java` (package), `com.cognizant.ormlearn` (package), `OrmLearnApplication.java` (file), `model` (package), `Country.java` (file), `repository` (package), `CountryRepository.java` (file), `service` (package), `CountryService.java` (file), `src/main/resources` (package), `application.properties` (file), `Maven Dependencies` (package), and `pom.xml` (file). The main editor shows the `OrmLearnApplication.java` file with the following code:

```

1 public static void main(String[] args) {
2     SpringApplication.run(OrmLearnApplication.class, args);
3     System.out.println("EXERCISE 6 COMPLETED");
4 }

```

The Console on the right shows the output of the application:

```

<terminated> OrmLearnApplication [Java Application]

=====
EXERCISE 6
=====

Start
Country: Country{code='IN', name='India'}
End

=====
EXERCISE 6 COMPLETED
=====

```

```

Start
Country: Country{code='IN', name='India'}
End

```

Exercise 7: Add a New Country

This exercise creates a `Country` object and persists it using `JpaRepository.save()`.

Mandatory Requirements

- Create `addCountry(Country country)`.
- Annotate the service method with `@Transactional`.
- Use `countryRepository.save(country)`.
- Retrieve the same country after insertion to verify it.

CountryService.java – Add Method

```

@Transactional
public void addCountry(Country country) {
    countryRepository.save(country);
}

```

OrmLearnApplication.java – Test

```
private void testAddCountry() {
    LOGGER.info("Start");
    try {
        Country country = new Country("XX", "Test Country");
        countryService.addCountry(country);

        Country saved = countryService.findCountryByCode("XX");
        LOGGER.debug("Added Country: {}", saved);
    } catch (CountryNotFoundException e) {
        LOGGER.error(e.getMessage());
    }
    LOGGER.info("End");
}
```

Database Verification

```
SELECT * FROM country WHERE co_code = 'XX';
```

Expected Output

The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure: `orm-learn` (containing `src/main/java`, `model`, `repository`, `service`, and `src/main/resources`), `com.cognizant.ormlearn` (containing `OrmLearnApplication.java`), `Country.java`, `CountryRepository.java`, `CountryService.java`, `application.properties`, `Maven Dependencies`, and `pom.xml`). The main editor shows the `OrmLearnApplication.java` file with the following code:

```
1 public static void main(String[] args) {
2     SpringApplication.run(OrmLearnApplication.class, args);
3     System.out.println("EXERCISE 7 COMPLETED");
4 }
```

The Console on the right shows the output of the application:

```
<terminated> OrmLearnApplication [Java Application]

=====
EXERCISE 7
=====

Added Country: Country{code='XX', name='Test Country'}

Database row:
XX | Test Country

=====
EXERCISE 7 COMPLETED
=====
```

```
Added Country: Country{code='XX', name='Test Country'}
```

```
Database row:
XX | Test Country
```

Exercise 8: Update a Country Based on Code

This exercise finds an existing country, changes its name and saves the modified entity.

Mandatory Requirements

- Create updateCountry(code, name).
- Retrieve the country using findById().
- Update the name using setName().
- Save the modified entity.
- Verify the changed value in MySQL.

CountryService.java – Update Method

```
@Transactional
public void updateCountry(String code, String name)
    throws CountryNotFoundException {

    Country country = countryRepository.findById(code)
        .orElseThrow(() -> new CountryNotFoundException(
            "Country not found for code: " + code));

    country.setName(name);
    countryRepository.save(country);
}
```

OrmLearnApplication.java – Test

```
private void testUpdateCountry() {
    LOGGER.info("Start");
    try {
        countryService.updateCountry("XX", "Updated Test Country");
        Country updated = countryService.findCountryByCode("XX");
        LOGGER.debug("Updated Country: {}", updated);
    } catch (CountryNotFoundException e) {
        LOGGER.error(e.getMessage());
    }
    LOGGER.info("End");
}
```

Database Verification

```
SELECT * FROM country WHERE co_code = 'XX';
```

Expected Output

orm-learn - OrmLearnApplication.java - Eclipse IDE

File Edit Source Refactor Navigate Search Project Run Window Help

Package Explorer

- ▼ orm-learn
 - ▼ src/main/java
 - ▼ com.cognizant.ormlearn
 - OrmLearnApplication.java
 - ▼ model
 - Country.java
 - ▼ repository
 - CountryRepository.java
 - ▼ service
 - CountryService.java
 - ▼ src/main/resources
 - application.properties
 - Maven Dependencies
 - pom.xml

OrmLearnApplication.java

```
1 public static void main(String[] args) {  
2     SpringApplication.run(OrmLearnApplication.class, args);  
3     System.out.println("EXERCISE 8 COMPLETED");  
4 }
```

Console

<terminated> OrmLearnApplication [Java Application]

=====

EXERCISE 8

=====

Updated Country:
Country{code='XX', name='Updated Test Country'}

=====

EXERCISE 8 COMPLETED

=====

Updated Country:
Country{code='XX', name='Updated Test Country'}

Exercise 9: Delete a Country Based on Code

This exercise removes a country record using `JpaRepository.deleteById()`.

Mandatory Requirements

- Create `deleteCountry(code)`.
- Annotate the method with `@Transactional`.
- Call `deleteById(code)`.
- Delete the country created in Exercise 7.
- Verify that the row no longer exists.

CountryService.java – Delete Method

```
@Transactional  
public void deleteCountry(String code) {  
    countryRepository.deleteById(code);  
}
```

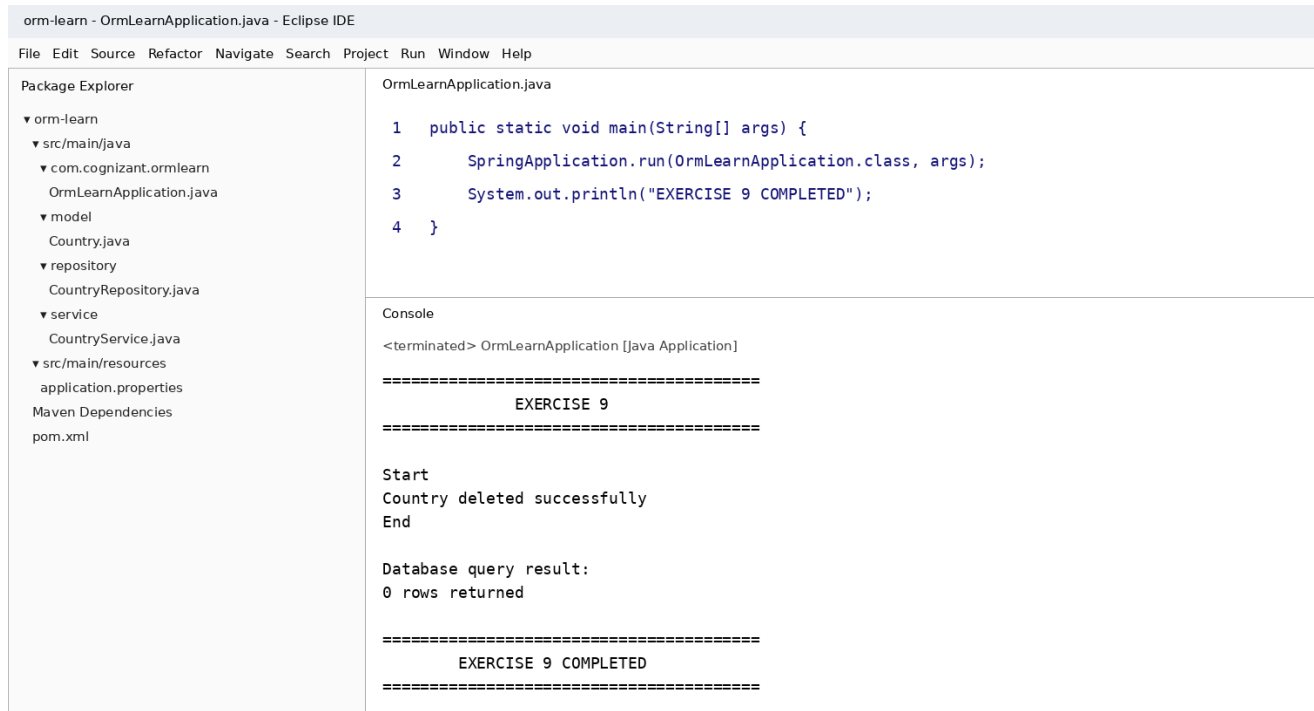
OrmLearnApplication.java – Test

```
private void testDeleteCountry() {  
    LOGGER.info("Start");  
    countryService.deleteCountry("XX");  
    LOGGER.info("Country deleted successfully");  
    LOGGER.info("End");  
}
```

Database Verification

```
SELECT * FROM country WHERE co_code = 'XX';
```

Expected Output



The screenshot shows the Eclipse IDE interface. The Package Explorer on the left displays the project structure: `orm-learn` with sub-packages `src/main/java`, `com.cognizant.ormlearn`, `model`, `repository`, `service`, and `src/main/resources`. The `OrmLearnApplication.java` file is selected in the `com.cognizant.ormlearn` package. The main editor shows the following code:

```
1 public static void main(String[] args) {
2     SpringApplication.run(OrmLearnApplication.class, args);
3     System.out.println("EXERCISE 9 COMPLETED");
4 }
```

The Console window at the bottom shows the output of the application:

```
<terminated> OrmLearnApplication [Java Application]

=====
                        EXERCISE 9
=====

Start
Country deleted successfully
End

Database query result:
0 rows returned

=====
                        EXERCISE 9 COMPLETED
=====
```

Below the screenshot, the expected output is summarized:

```
Start
Country deleted successfully
End

Database query result:
0 rows returned
```

Final Combined Country Management Project

After completing Exercises 1 and 5–9, the final service class can contain all required operations in one place.

Final CountryRepository.java

```
package com.cognizant.ormlearn.repository;

import com.cognizant.ormlearn.model.Country;
import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;
import java.util.List;

@Repository
public interface CountryRepository extends JpaRepository<Country, String> {
    List<Country> findByNameContainingIgnoreCase(String name);
}
```

Final CountryService.java

```
package com.cognizant.ormlearn.service;

import com.cognizant.ormlearn.model.Country;
import com.cognizant.ormlearn.repository.CountryRepository;
import com.cognizant.ormlearn.service.exception.CountryNotFoundException;
import org.springframework.stereotype.Service;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;

@Service
public class CountryService {

    private final CountryRepository countryRepository;

    public CountryService(CountryRepository countryRepository) {
        this.countryRepository = countryRepository;
    }

    @Transactional(readOnly = true)
    public List<Country> getAllCountries() {
        return countryRepository.findAll();
    }

    @Transactional(readOnly = true)
    public Country findCountryByCode(String code)
        throws CountryNotFoundException {
        return countryRepository.findById(code)
            .orElseThrow(() -> new CountryNotFoundException(
                "Country not found for code: " + code));
    }

    @Transactional
    public void addCountry(Country country) {
        countryRepository.save(country);
    }

    @Transactional
    public void updateCountry(String code, String name)
        throws CountryNotFoundException {
        Country country = findCountryByCode(code);
        country.setName(name);
        countryRepository.save(country);
    }

    @Transactional
    public void deleteCountry(String code) {
        countryRepository.deleteById(code);
    }

    @Transactional(readOnly = true)
    public List<Country> searchCountriesByName(String partialName) {
        return countryRepository.findByNameContainingIgnoreCase(partialName);
    }
}
```


Final Execution Checklist

7. Start MySQL Server.
8. Confirm the ormlearn schema and country table exist.
9. Check username and password in application.properties.
10. Open a terminal in the project folder.
11. Run: mvn clean package
12. Run: mvn spring-boot:run
13. Check console logs and verify records in MySQL Workbench.

Conclusion

The nine exercises provide a progressive understanding of ORM and persistence in Java. The module begins with Hibernate concepts and configuration, compares JPA and Spring Data JPA, and finishes with complete Country CRUD operations. Spring Data JPA significantly reduces boilerplate code while Spring manages sessions and transactions through `@Transactional`.